Rapport

Autocomplétion et structures de données

Structures de données — IFT2015

Par

Joseph Soufsaf
Parsa Makari

Présenté à

François Major

7 avril 2026

Université de Montréal

Répondez aux quatre questions ci-dessous.
Les réponses doivent utiliser une ***notation asymptotique précise (Big-O)***
et être accompagnées d'une justification claire.

Q1 — Invariants des structures de données (12 pts)

Pour chacune des trois structures de données utilisées dans ce projet, définissez les **invariants structurels** (propriétés qui doivent toujours être vraies) et expliquez comment les opérations principales maintiennent ces invariants.

1a. Table de fréquences (`HashMap`) (4 pts)

Invariants structurels :

- Les données sont stockées en paires `<clé, valeur>`, où la valeur est la fréquence de la clé
- Clés uniques (invariant d'une Map)
- Valeurs (fréquences) positives
- Insertion et accès aux éléments du tableau selon l'algorithme de hachage
- Gestion des collisions entre deux clés par une des méthodes possibles du HashMap

Maintenance des invariants :

- Insertion/mise à jour → préserve unicité des clés (si la clé existe on augmente sa fréquence, sinon on ajoute la clé avec fréquence = 1)
- Suppression → décrémente et supprime si fréquence = 0, évite les valeurs négatives
- Accès → lecture seule, tous les invariants demeurent inchangés
- Hachage → résout les collisions, maintient l'intégrité de la structure

1b. Arbre de préfixes (`PrefixTrie`) (4 pts)

Invariants structurels :

- Une racine unique
- Un chemin représente un préfixe

— Les nœuds qui complètent un mot sont marqués comme tels
— Chaque mot est représenté par un seul chemin unique

Maintenance des invariants :

— Les opérations d'insertion, suppression et recherche n'affectent pas les invariants
— La recherche ne modifie pas la structure de l'arbre

1c. Tas min (`PriorityQueue`) (4 pts)

Invariants du min-heap :

— Le parent est toujours plus petit que ses enfants
— C'est un arbre complet (rempli de gauche à droite)
— La racine est l'élément le plus petit

Maintenance des invariants :

— **Insertion :** L'élément est ajouté à la fin, puis remonté (*sift-up*) si plus petit que son parent
— **Suppression :** La racine est remplacée par le dernier élément et descendue (*sift-down*) jusqu'à satisfaire la propriété de min-heap
— **Recherche :** Ne modifie pas la structure du heap, donc l'invariant est maintenu

Q2 — Complexité asymptotique de l'entraînement (12 pts)

Considérez un corpus contenant N tokens. Le vocabulaire a une taille V ($V \leq N$). La longueur moyenne des tokens est supposée constante.

Analysez la complexité de la méthode `train(List<List<String» sentences)` en décomposant chaque étape.

Variables à utiliser : N (tokens), V (vocabulaire).

2a. Construction de la table de fréquences (`unigrams`) (4 pts)

On utilise une HashMap pour stocker N tokens.

Complexité temporelle : Puisque dans une HashMap, l'insertion ou la recherche se fait en $\mathcal{O}(1)$ pour chaque élément, et nous avons N tokens, la complexité est $\mathcal{O}(N)$.

Complexité spatiale : L'espace occupé est au maximum $\mathcal{O}(V)$, car dans une HashMap il n'y a pas de duplication d'éléments. Au pire cas, chaque token unique occupe une entrée.

2b. Construction du trie (4 pts)

Un trie doit passer à travers chaque lettre du mot pendant son insertion.

Complexité temporelle : Si un mot a une longueur moyenne de M (supposée constante) et nous avons N tokens, l'arbre prend $\mathcal{O}(N \times M) = \mathcal{O}(N)$ pour être construit (puisque M est constant).

Complexité spatiale : Le trie prend $\mathcal{O}(M \times V) = \mathcal{O}(V)$ espace, car on stocke chaque lettre pour chaque token unique.

2c. Construction des tables bigrammes et trigrammes (4 pts)

On utilise une HashMap pour stocker des paires (bigrammes) et des triplets (trigrammes) de tokens.

Nombre d'éléments : Avec N tokens, il y a $N-1$ paires et $N-2$ triplets à traiter.

Complexité temporelle : L'insertion et la recherche en HashMap se font en $\mathcal{O}(1)$. Pour $N-1$ et $N-2$ éléments, la complexité asymptotique est $\mathcal{O}(N)$.

Complexité spatiale :

- Bigrammes : $\min(\mathcal{O}(V^2), \mathcal{O}(N))$ (le minimum entre le nombre maximum de combinaisons de V mots et la longueur du corpus)
- Trigrammes : $\min(\mathcal{O}(V^3), \mathcal{O}(N))$

Q3 — Complexité asymptotique des requêtes (10 pts)

Considérez une requête depuis un modèle déjà entraîné sur un vocabulaire de taille V. On demande les k meilleures suggestions pour un contexte donné. Le préfixe soumis à `complete` a une longueur L.

Variables à utiliser : V, k, L.

3a. `topK` via la table de fréquences (4 pts)

Phase 1 — Parcours : On doit parcourir tous les mots de la table pour trouver ceux ayant le préfixe, ce qui prend $\mathcal{O}(V)$.

Phase 2 — Sélection du top-K : On utilise un min-heap des fréquences avec taille maximale K. Pour chacun des V mots :

- Si le heap n'est pas plein : insertion en $\mathcal{O}(\log K)$
- Si le heap est plein et le mot a une meilleure fréquence que la racine : suppression + insertion en $\mathcal{O}(\log K)$
- Sinon : on passe

Cette phase coûte $\mathcal{O}(V \log K)$ au pire cas.

Complexité totale :

$$\mathcal{O}(V) + \mathcal{O}(V \log K) = \mathcal{O}(V \log K)$$

3b. `complete` via le trie (6 pts)

Exprimer la complexité en distinguant la phase de descente au préfixe et la phase de DFS + sélection top-k.

Phase 1 — Descente au préfixe : On parcourt les L nœuds du trie correspondant au préfixe, ce qui prend $\mathcal{O}(L)$.

Phase 2 — DFS pour récupérer tous les mots : On utilise une recherche DFS à partir du nœud final du préfixe pour trouver tous les mots qui incluent ce préfixe. Au pire cas, on explore tout le sous-arbre du trie, ce qui est $\mathcal{O}(N)$ (tous les nœuds).

Phase 3 — Sélection du top-K : On emploie la même stratégie qu'en 3a pour gérer les top-K, ce qui coûte $\mathcal{O}(V \log k)$.

Complexité totale :

$$\mathcal{O}(L) + \mathcal{O}(N) + \mathcal{O}(V \log k) = \mathcal{O}(L + N + V \log k)$$

Q4 — Comparaison d'implémentations alternatives (6 pts)

Choisissez **une** des structures de données utilisées dans ce projet et comparez-la à une alternative plausible.

Exemples : trie vs tableau trié, tas min vs tri complet,
table de hachage vs arbre binaire de recherche équilibré.

Votre réponse doit inclure :

- une description des deux approches,
- la complexité des opérations principales pour chacu :ne,
- une discussion sur l'utilisation mémoire,
- une conclusion sur le contexte où chaque approche serait préférable.

Comparaison HashMap vs. ABR Équilibré :

Si on avait décidé d'employer un arbre binaire de recherche au lieu d'un HashMap pour l'implémentation de notre table de fréquences, voici ce qui changerait :

En supposant que nous conservons les mots comme clés et les fréquences comme valeurs :

L'insertion, l'accès et la suppression deviendraient en $\mathcal{O}(\log V)$ au lieu de $\mathcal{O}(1)$.

Cependant, puisque l'ABR impose un ordre total, on peut trouver le premier mot avec un préfixe en $\mathcal{O}(\log V)$, puis parcourir pour trouver tous les mots avec ce préfixe (pour s mots : $\mathcal{O}(\log V + s)$).

Ainsi, on obtient les avantages d'une map ordonnée pour les requêtes par intervalle :

Cela est intéressant si les requêtes par préfixe et parcours ordered sont fréquents. Mais pour des insertions/accès fréquents, le HashMap demeure meilleur car il offre $\mathcal{O}(1)$ vs $\mathcal{O}(\log V)$.

Recommandation : En autocomplétion, le HashMap est préférable car le trie gère déjà les requêtes par préfixe efficacement.

Bibliographie

Les références doivent être présentées selon le style **APA**.

Exemple :

— Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Java* (6e éd.). Wiley.

Ajoutez ci-dessous toutes les sources utilisées :

—

—

—

—